

Gibbs Sampler for Email Classification

Patrick Mellady

Here is an implementation of a Gibbs sampler to email spam/not-spam data to perform classification.

Distributional Assumptions

We start by assuming distributional properties about our data. We make the assumption that emails are drawn from a mixture of two poisson distributions. Let Y_i be the number of “spam” words in the i^{th} email, then we are assuming

$$Y_i \sim (1 - q) \cdot \text{pois}(\lambda_0) + q \cdot \text{pois}(\lambda_1)$$

In this situation, the λ_0 distribution represents the count of “spam” words in spam emails and the λ_1 distribution represents the count of “spam” words in emails that are not spam. When we say “spam” words, we mean words that are typically seen in spam email.

We introduce a latent variable, Z_i , which labels each email as spam or not. Using the notation that $\boldsymbol{\lambda} = (\lambda_0, \lambda_1)$, $\mathbf{Y} = (Y_1, Y_2, \dots, Y_n)$, and $\mathbf{Z} = (Z_1, \dots, Z_n)$, we have the following hierarchical structure

$$\begin{aligned} Y_i | q, \boldsymbol{\lambda}, Z_i &\sim \text{pois}(\lambda_{Z_i}) \\ Z_i | q &\sim \text{bern}(q) \\ q &\sim \text{beta}(a, b) \\ \lambda_0, \lambda_1 &\overset{iid}{\sim} \text{exp}(\beta) \quad \text{where } \beta \text{ is the exponential scale parameter} \end{aligned}$$

Distributional Derivations

Starting with the hierarchical model stated above, we derive each of the conditional distributions required in the model. We start with the simplest

Distribution of $\mathbf{Y} | q, \boldsymbol{\lambda}, \mathbf{Z}$

Since Z_i identifies the distribution from which the email was drawn, we have that

$$\begin{aligned} Y_i &\sim \text{pois}(\lambda_1) \text{ if } Z_i = 1 \\ Y_i &\sim \text{pois}(\lambda_0) \text{ if } Z_i = 0 \end{aligned}$$

Distribution of $\lambda_i | \mathbf{Y}, q, \mathbf{Z}$

Let us start with λ_0 . We can write

$$P(\lambda_0 | \lambda_1, \mathbf{Y}, q, \mathbf{Z}) = P(\lambda_0 | \lambda_1, \mathbf{Y}, \mathbf{Z}) = c\pi(\lambda_0) \prod_{i=1}^n f(y_i | \lambda_0)^{1-z_i} f(y_i | \lambda_1)^{z_i}$$

This is a straightforward application of Bayes' rule where the likelihood of the data is given by $\prod_{i=1}^n f(y_i | \lambda_0)^{1-z_i} f(y_i | \lambda_1)^{z_i}$, the prior density is $\pi(\lambda_0)$ and the constant c is for normalization. Since we assume an exponential distribution on λ_0 and since $f(y_i | \lambda_1)^{z_i}$ has no dependence on λ_0 , we can simplify this as follows:

$$P(\lambda_0 | \lambda_1, \mathbf{Y}, q, \mathbf{Z}) = c_1 \frac{1}{\beta} e^{-\lambda_0/\beta} \prod_{i=1}^n f(y_i | \lambda_0)^{1-z_i} = c_1 \frac{1}{\beta} e^{-\lambda_0/\beta} \prod_{i=1}^n \left(\frac{e^{-\lambda_0} \lambda_0^{y_i}}{y_i!} \right)^{1-z_i}$$

We can further simplify this by rearranging more constants to obtain

$$P(\lambda_0 | \lambda_1, \mathbf{Y}, q, \mathbf{Z}) = c_2 \lambda_0^{\sum y_i (1-z_i)} e^{-\lambda_0(\frac{1}{\beta} + \sum (1-z_i))}$$

This all tells us that

$$\lambda_0 | \lambda_1, \mathbf{Y}, q, \mathbf{Z} \sim \text{gamma}(\sum y_i (1-z_i) + 1, (\frac{1}{\beta} + \sum (1-z_i))^{-1})$$

Similarly, by symmetry, we have

$$\lambda_1 | \lambda_0, \mathbf{Y}, q, \mathbf{Z} \sim \text{gamma}(\sum y_i z_i + 1, (\frac{1}{\beta} + \sum z_i)^{-1})$$

The distribution of $q | \mathbf{Y}, \mathbf{Z}, \lambda$

Note that $P(q | \mathbf{Y}, \mathbf{Z}, \lambda) = P(q | \mathbf{Z}) = p(\mathbf{Z} | q)P(q)$. With the conditional distribution of $Z_i | q$ as in the model statement and with the assigned $\text{beta}(a, b)$ prior on q , this gives us

$$P(q | \mathbf{Y}, \mathbf{Z}, \lambda) = cq^{a-1}(1-q)^{b-1} \prod_{i=1}^n q^{z_i} (1-q)^{1-z_i} = cq^{a+\sum z_i-1} (1-q)^{b+n-\sum z_i-1}$$

and hence $q | \mathbf{Y}, \mathbf{Z}, \lambda \sim \text{beta}(a + \sum z_i, b + n - \sum z_i)$.

The distribution of $\mathbf{Z} | q, \mathbf{Y}, \lambda$

Lastly, since we will be sampling the vector of Z_i s simultaneously, we need to find the posterior distribution of $\mathbf{Z} | q, \mathbf{Y}, \lambda$. To do this, note that

$$P(\mathbf{Z} | q, \mathbf{Y}, \lambda) = cP(\mathbf{Z}, q, \mathbf{Y}, \lambda) = cP(\mathbf{Y} | q, \mathbf{Z}, \lambda)P(q, \mathbf{Z}) = cP(\mathbf{Y} | q, \mathbf{Z}, \lambda)P(\mathbf{Z} | q)P(q)$$

Since the prior distribution of q is seen as constant in the distribution of $\mathbf{Z}$ is can be absorbed into the constant to obtain

$$P(\mathbf{Z} | q, \mathbf{Y}, \lambda) = c_1 P(\mathbf{Y} | q, \mathbf{Z}, \lambda) P(\mathbf{Z} | q) = c_1 P(\mathbf{Y} | \mathbf{Z}, \lambda) P(\mathbf{Z} | q)$$

Where we simplify $P(\mathbf{Y}|q, \mathbf{Z}, \boldsymbol{\lambda})$ to $P(\mathbf{Y}|\mathbf{Z}, \boldsymbol{\lambda})$ since the condition on $\mathbf{Z}$ makes q superfluous. Now, since the Y_i s and Z_i s are conditionally independent, we can write this as a product of the mass functions as follows

$$P(\mathbf{Z}|q, \mathbf{Y}, \boldsymbol{\lambda}) = c_1 \prod_{i=1}^n f(y_i|\lambda_0)^{1-z_i} f(y_i|\lambda_1)^{z_i} q^{z_i} (1-q)^{1-z_i}$$

We now let $p_{0i} = f(y_i|\lambda_0)(1-q)$ and $p_{1i} = f(y_i|\lambda_1)q$, which gives

$$P(\mathbf{Z}|q, \mathbf{Y}, \boldsymbol{\lambda}) = c_1 \prod_{i=1}^n p_{0i}^{1-z_i} p_{1i}^{z_i}$$

Defining $p = \frac{p_{1i}}{p_{1i}+p_{0i}}$ and simplifying the above gives

$$P(\mathbf{Z}|q, \mathbf{Y}, \boldsymbol{\lambda}) = c_2 \prod_{i=1}^n (1-p)^{1-z_i} p^{z_i} = c_2 \prod_{i=1}^n f(z_i|p)$$

So, we obtain

$$Z_i|q, \mathbf{Y}, \boldsymbol{\lambda} \sim \text{bern}(p), \quad \text{where } p = \frac{f(y_i|\lambda_1)q}{f(y_i|\lambda_1)q + f(y_i|\lambda_0)(1-q)}$$

Implementation

With the above distributions, we can perform the iterative sampling algorithm as stated from before. We are using the `spam.csv` data set. Here is where we return to the idea of “spam words”.

We use the training data to select some words commonly seen in spam emails. We will not use every word found in spam emails (because spam emails can contain words that are used in non-spam emails) but we will select the words that are highly unlikely to show up in non-spam emails. This can be seen in the code below.

```
# Reading the data
spam<-read.csv("spam.csv")
n<-length(spam[,1])
real<-rep(1,n)
real[spam[,1]=="spam"]<-0

# Create a list of spam words to test each email against
spam_words<-tolower(c("Free", "entry", "wkly", "comp", "win", "final", "tkts",
                      "Text", "entry", "FreeMsg", "darling", "fun", "XxX",
                      "entitled", "Update", "credit", "claim", "subscription",
                      "special", "pleased", "valued", "jackpot", "prize"))

# Assigning spam word counts to each email
# Create X vector representing the number of spam words in each email
X<-rep(0,n)

# Search through each email to see update spam word count
vec<-spam_words
for(word in vec){
  for(i in 1:n){
```

```

    if(grepl(word, tolower(spam[i,2]))){
      X[i]<-X[i]+1
    }
  }
}

```

As shown above, the commonly seen spam words are stored in a vector. We then use these words to assign a count of all the spam words in any particular email. Now, we will use the distributional model from above to run a Gibbs sampler and classify the emails.

```

Z0<-rbinom(n, 1, .5)      # Initial value of Z for Gibbs Sampler
beta<-1                   # Used in exponential prior
a<-1                      # Used in beta prior
b<-1                      # Used in beta Prior
m<-5000                   # Number of Gibbs Sampler iterations

# Function to make easy sampling
samp<-function(X, l0, l1, p){
  p.0<-(1-p)*dpois(X, l0)
  p.1<-p*dpois(X, l1)
  q<-p.1/(p.1+p.0)
  return(rbinom(length(X), 1, q))
}

# Running the Gibbs Sampler
## Initialize sample containers
Z<-vector(mode='list', m)
lambda<-vector("list", m)
mix_param<-vector("numeric", m)
l1<-vector("numeric", m)

## Set initial parameter values
Z[[1]]<-Z0
lambda[[1]]<-c(1,1)
mix_param[1]<-0.5
l1[1]<-sum(log(.5*dpois(X, 1)+0.5*dpois(X, 1)))

for(j in 2:m){
  ## Sample the rate and mixing parameters
  Z_j<-Z[[j-1]]
  l0<-rgamma(1,sum(X*(1-Z_j))+1, scale=beta/(beta*sum(1-Z_j)+1))
  l1<-rgamma(1,sum(X*Z_j)+1, scale=beta/(beta*sum(Z_j)+1))
  q<-rbeta(1, a+sum(Z_j), b+sum(1-Z_j))
  lambda[[j]]<-c(l0, l1)
  mix_param[j]<-q
  Z[[j]]<-samp(X, l0, l1, q)

  ## Perform label alignment check and switch if necessary
  if(l1>l0){
    lambda[[j]]<-rev(lambda[[j]])
    mix_param[j]<-1-mix_param[j]
    Z[[j]]<-1-Z[[j]]
  }
}

```

```

# Calculate and track log likelihood
ll[j]<-sum(log((1-mix_param[j])*dpois(X, lambda[[j]][1])+
      mix_param[j]*dpois(X, lambda[[j]][2])))
}

```

With the sampling done, we can now use the resulting list of $\mathbf{Z}$ to make a decision rule about whether or not an email is spam.

```

# Finding the posterior mean of Z (post burn-in)
burnin_frac<-.75
post_idx<-floor(burnin_frac*m):m

Z.bar<-rep(0,n)
for(i in post_idx){
  Z.bar<-Z.bar+Z[[i]]
}
Z.bar<-Z.bar/length(post_idx)

# Find posterior mean of parameters
lambda_post<-do.call(rbind, lambda[post_idx])
l0_hat<-mean(lambda_post[, 1])
l1_hat<-mean(lambda_post[, 2])
q_hat<-mean(mix_param[post_idx])

# Rounding the estimates to binary spam/not-spam labels
Z_prob<-Z.bar
Z.bar<-round(Z.bar)

```

We can now check how our model fits by performing some error analysis with the true classification of the emails.

```

# Error Analysis
# Finding the accuracy of the algorithm
correct<-sum(Z.bar==real)
percent_correct<-correct/n
tabs<-table(Z.bar, real, dnn=list("Estimated", "True"))
kable(tabs, row.names = TRUE,
      caption=paste0("Overall Accuracy: ", round(percent_correct, 3)))

```

Table 1: Overall Accuracy: 0.91

	0	1
0	266	18
1	481	4807

We can also see what our parameter estimates are as given in the table below

```

# Determining the real values of the parameters of our model
l0_real<-mean(X[real==0])

```

```

l1_real<-mean(X[real==1])
q_real<-sum(real)/n

parameter_summary<-round(data.frame(Truth=c(l0_real, l1_real, q_real),
                                         Estimates=c(l0_hat, l1_hat, q_hat)),3)
rownames(parameter_summary)<-c("Lambda 0", "Lambda 1", "q")
kable(round(parameter_summary,3), row.names = TRUE,
      caption="Model Parameters and their Estimates")

```

Table 2: Model Parameters and their Estimates

	Truth	Estimates
Lambda 0	1.222	1.360
Lambda 1	0.090	0.094
q	0.866	0.882

Using the above estimates, we can see which emails the model has difficulty classifying.

```

# Finding the spam word count of the mislabeled emails
missed<-which(real!=Z.bar)
l_missed<-mean(X[missed])
l_01<-(l0_real+l1_real)/2

missed_df<-round(data.frame(Count=length(missed),
                             `Avg. Spam Words in Missed Emails`=l_missed,
                             `Avg. of True Avg. Spam Words`=l_01,
                             check.names = FALSE), 3)
kable(missed_df, caption="Average Spam Words in Misclassified Emails")

```

Table 3: Average Spam Words in Misclassified Emails

Count	Avg. Spam Words in Missed Emails	Avg. of True Avg. Spam Words
499	0.553	0.656

These calculations show that the misclassified emails had an average spam word count of 0.553, which is incredibly close to the average of the average spam word count between the two groups, 0.656

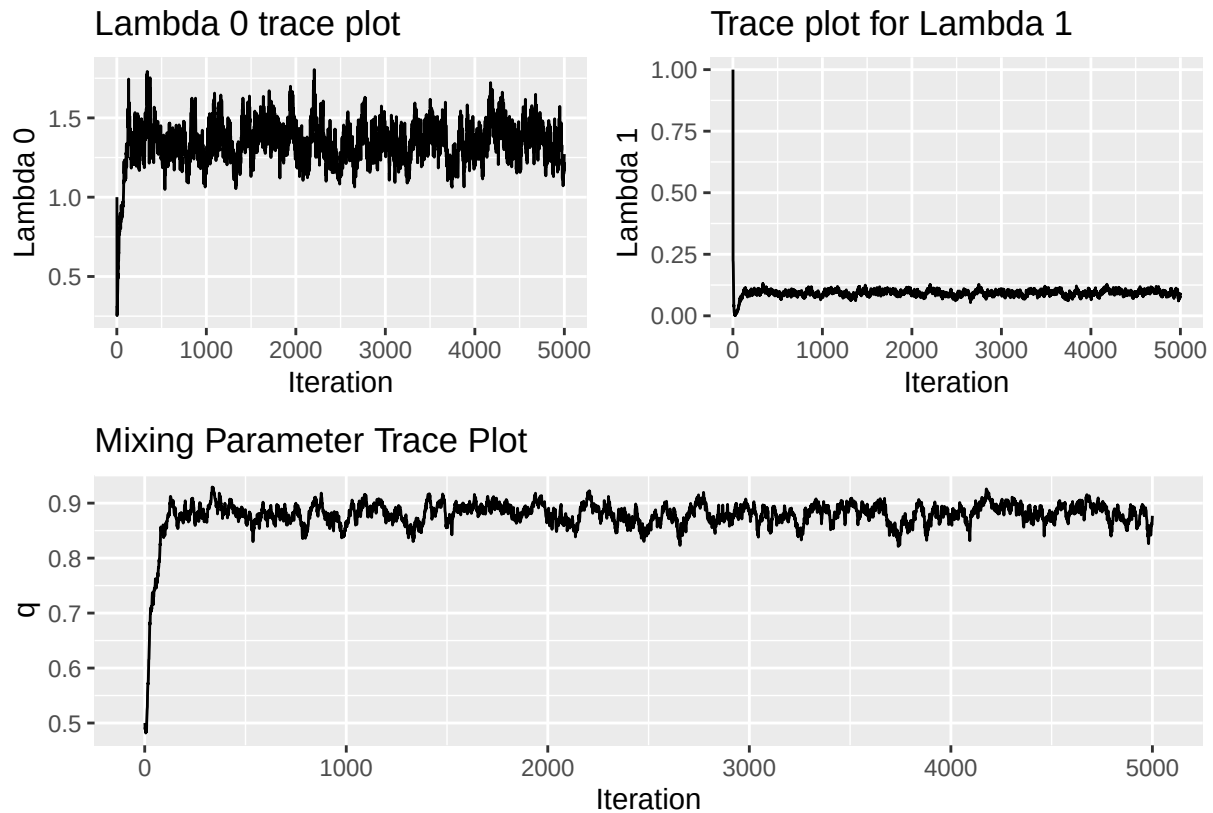
We can examine the MCMC chain convergence with the following trace plots for the model parameters:

```

L<-as.data.frame(do.call(rbind, lambda))
colnames(L)<-c("l0", "l1")
L<-cbind(Iteration=1:m, L)

((ggplot(L)+geom_line(aes(x=Iteration, y=l0))+
  labs(title="Lambda 0 trace plot")+ylab("Lambda 0")) |
  (ggplot(L)+geom_line(aes(x=Iteration, y=l1))+
    labs(title="Trace plot for Lambda 1")+ylab("Lambda 1")) /
  (ggplot()+geom_line(aes(x=1:m, y=mix_param))+
    labs(title="Mixing Parameter Trace Plot")+xlab("Iteration")+ylab("q"))

```



Lastly, we plot the misclassification rate and log likelihood with respect to the iteration number to see how the model performs as the iterations increase.

```
# Finding and plotting the error at each iteration of the Gibbs Sampler
error_vec<-c()
for(i in 1:m){
  error_vec<-c(error_vec, mean(Z[[i]]!=real))
}

for_plot<-data.frame(Iteration=1:m,
                     Error=error_vec,
                     LL=ll)

p1<-ggplot(for_plot)+geom_line(aes(x=Iteration, y=Error))+
  labs(title="Error Rate")
p2<-ggplot(for_plot)+geom_line(aes(x=Iteration, y=LL))+
  labs(title="Log Likelihood")+
  ylab("Log Likelihood")

p1 | p2
```

